Rejestr zarejestrowanych pojazdów

Grzegorz

Kozłowski

INF gr.1 sek.1

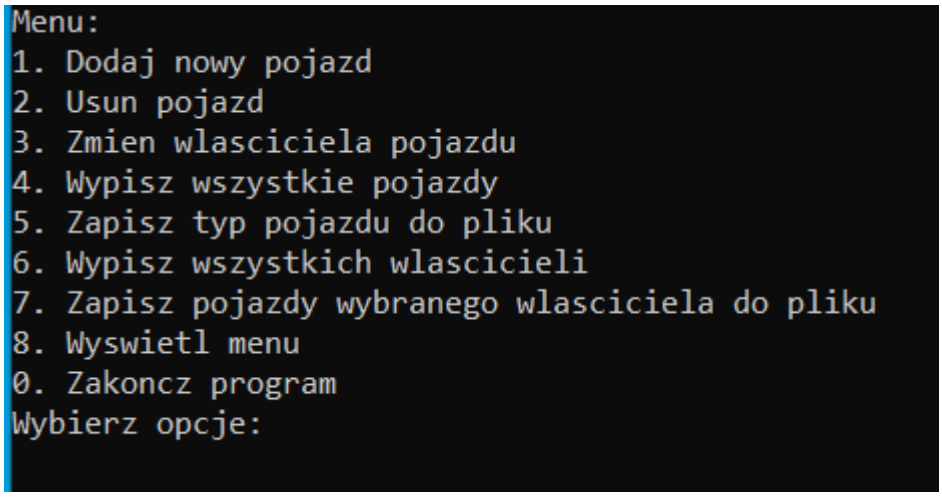
1. Ogólny opis programu:

Program „Rejestr Pojazdów” umożliwia zarządzanie i rejestrację różnorodnych pojazdów oraz ich właścicieli. Głównym celem programu jest śledzenie i zarządzanie informacjami o pojazdach, takimi jak samochody osobowe, motocykle oraz samochody ciężarowe, wraz z danymi o ich właścicielach. Pozwala nam na wykonywanie takich operacji: Dodawanie nowego pojazdu, usuwanie pojazdu, zmiana właściciela pojazdu, wypisywanie na ekran właścicieli oraz aktualnie zarejestrowanych pojazdów, zapisywanie do pliku tekstowego określonego typu pojazdów lub wszystkich pojazdów danego właściciela.

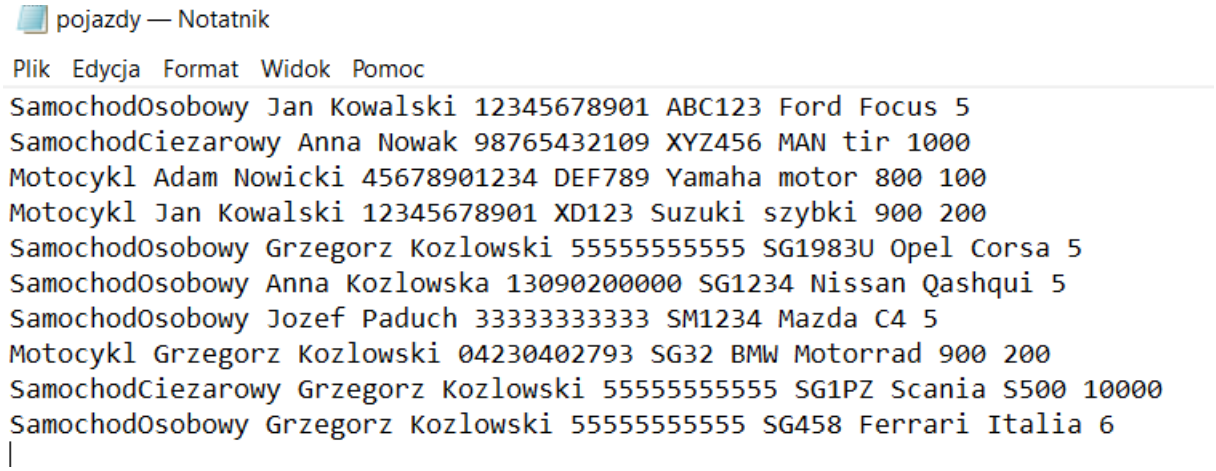
Program odporny jest na błędy danych wejściowych użytkownika.

2. Opis interfejsu zewnętrznego + testowanie

Program obsługuje się za pomocą menu. W którym użytkownik może wybierać jaką operacje chce wykonać.



Plik przechowujący dane które są wczytywane na początku wygląda w taki sposób:



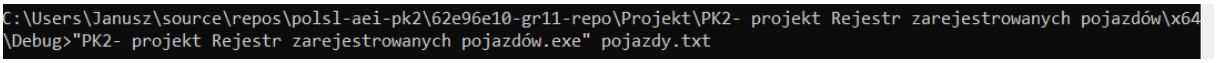
Dane przechowywane są w taki sposób, że:

Typ >> imię właściciela >> nazwisko >> pesel >> nr Rejestracyjny >> marka >> nazwa >> określona cecha(w przypadku motocyklu dwie)

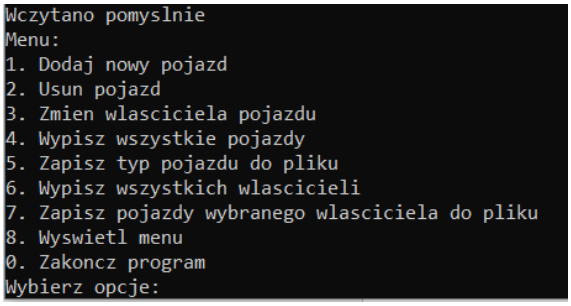
Po zakończeniu pracy programu w taki sam sposób uzupełnione o nowy stan wprowadzane do tego samego pliku.

Testowanie:

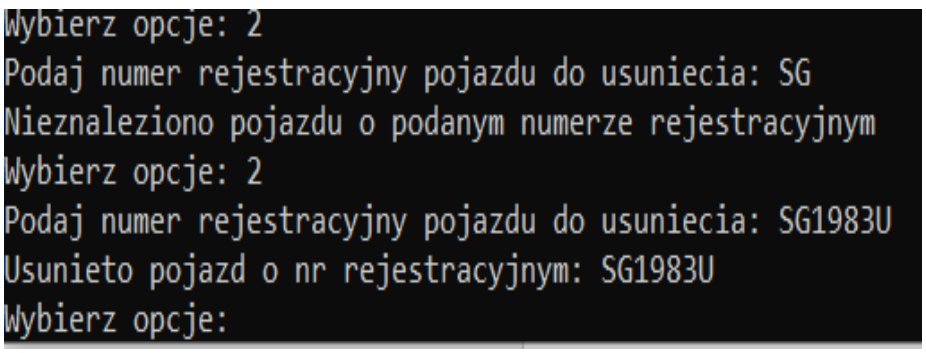
1. Uruchamiamy program w konsoli podając jako argument nazwę sformatowanego pliku tekstowego (u mnie pojazdy.txt)

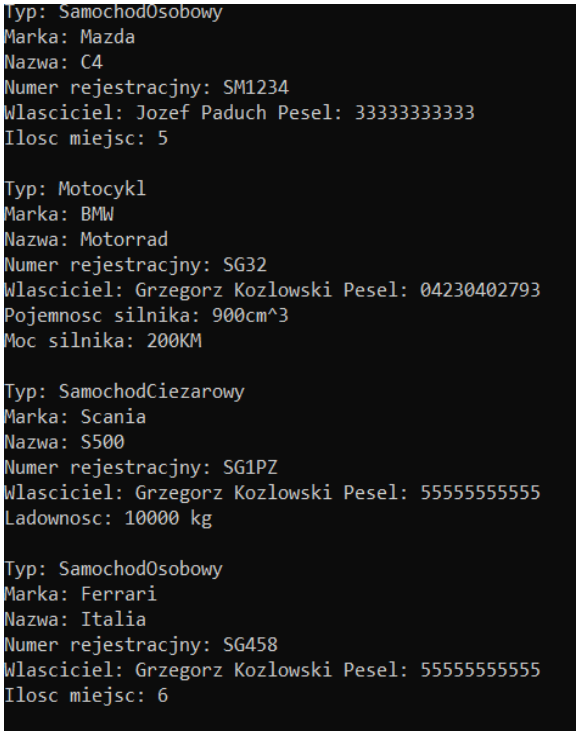


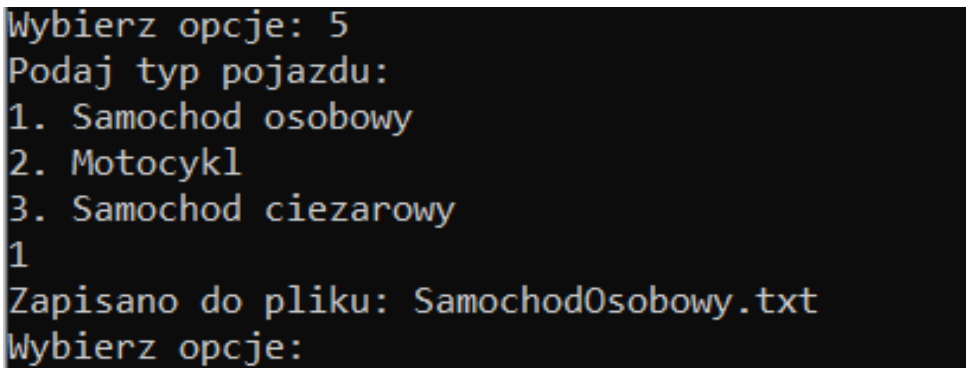
2. Program zwróci nam wiadomość, że wczytano plik pomyślnie- wybieramy dowolną funkcje z wyświetlonego menu.



3. Przykład funkcje nr 2, 4, 5:





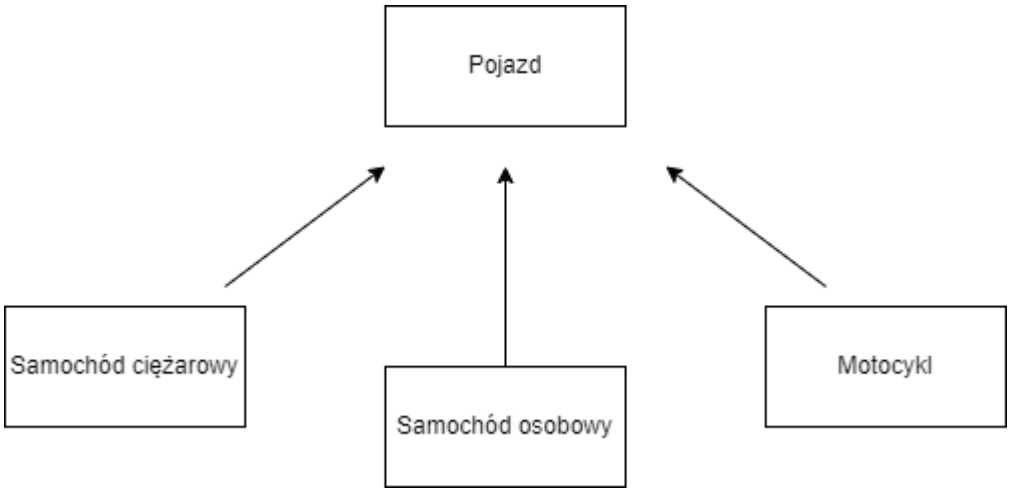


4. Zakończymy działanie programu wybierając 0 w menu. Aktualny stan danych zostanie zapisany do pliku podanego podczas uruchamiania programu.

3. Opis interfejsu wewnętrznego (wygenerowany za pomocą doxygena).

Istnieje 6 klas: Właściciel, Rejestr pojazdów, Pojazd, Samochód osobowy, Motocykl, Samochód ciężarowy.

Diagram pokazujący hierarchię klasy bazowej i jej pochodnych:



Pliki .cpp zostaną dodane na końcu sprawozdania

Dokumentacja klasy RejestrPojazdow

Metody publiczne

- void **dodajPojazd** (**Pojazd** *pojazd)
  Dodaje pojazd do rejestru.

- void **usunPojazd** (const std::string &nrRejestracyjny)
  Usuwa pojazd z rejestru na podstawie numeru rejestracyjnego.

- void **wyswietlPojazdy** () const
  Wyświetla wszystkie pojazdy w rejestrze.

- **Wlasciciel * znajdzWlasciciela** (const std::string &pesel)
  Znajduje właściciela na podstawie numeru PESEL.

- void **zapiszPojazdyTypu** (const std::string &typ, const std::string &**nazwaPliku**)
  Zapisuje pojazdy określonego typu do pliku.

- void **zapiszPojazdyWlasciciela** (const std::string &pesel, const std::string &**nazwaPliku**)
  Zapisuje pojazdy należące do danego właściciela do pliku.

- **~RejestrPojazdow** ()
  *Destruktor klasy **RejestrPojazdow**.*

- void **wypiszWlascicieli** () const
  Wyświetla listę właścicieli.

- bool **dodajWlasciciela** (const **Wlasciciel** &wlasciciel)
  Dodaje nowego właściciela do rejestru.

- bool **zmienWlasciciela** (const std::string &nrRejestracyjny, const **Wlasciciel** &nowyWlasciciel)
  Zmienia właściciela pojazdu.

- const std::vector< **Pojazd** * > & **getPojazdy** () const
  *Zwraca wektor wskaźników na obiekty typu **Pojazd**.*

Atrybuty publiczne

- std::string **nazwaPliku**
  Nazwa pliku do wczytania danych.

Atrybuty prywatne

- std::vector< **Pojazd** * > **pojazdy**
  *Wektor wskaźników na obiekty typu **Pojazd**.*

- std::vector< **Wlasciciel** > **wlasciciele**
  *Wektor obiektów typu **Wlasciciel**.*

Opis szczegółowy

Klasa reprezentująca rejestr pojazdów.
Definicja w linii **19** pliku **RejestrPojazdów.h**.

Dokumentacja konstruktora i destruktora

RejestrPojazdow::~RejestrPojazdow ()

Destruktor klasy **RejestrPojazdow**.
Definicja w linii **27** pliku **RejestrPojazdów.cpp**.

Dokumentacja funkcji składowych

*void RejestrPojazdow::dodajPojazd (Pojazd * pojazd)*

Dodaje pojazd do rejestru.

Parametry

pojazd	Wskaźnik na obiekt typu **Pojazd**.

Definicja w linii **5** pliku **RejestrPojazdów.cpp**.

bool RejestrPojazdow::dodajWlasciciela (const Wlasciciel & wlasciciel)

Dodaje nowego właściciela do rejestru.

Parametry

wlasciciel	Obiekt typu **Wlasciciel** do dodania.

Zwraca

true jeśli właściciel został dodany, false jeśli już istnieje.

Definicja w linii **45** pliku **RejestrPojazdów.cpp**.

*const std::vector< Pojazd * > & RejestrPojazdow::getPojazdy () const*

Zwraca wektor wskaźników na obiekty typu **Pojazd**.

Zwraca

Wektor wskaźników na obiekty typu **Pojazd**.

Definicja w linii **65** pliku **RejestrPojazdów.cpp**.

void RejestrPojazdow::usunPojazd (const std::string & nrRejestracyjny)

Usuwa pojazd z rejestru na podstawie numeru rejestracyjnego.

Parametry

nrRejestracyjny	Numer rejestracyjny pojazdu do usunięcia.

Definicja w linii **8** pliku **RejestrPojazdów.cpp**.

void RejestrPojazdow::wypiszWlascicieli () const

Wyświetla listę właścicieli.

Definicja w linii **40** pliku **RejestrPojazdów.cpp**.

void RejestrPojazdow::wyswietlPojazdy () const

Wyświetla wszystkie pojazdy w rejestrze.

Definicja w linii **22** pliku **RejestrPojazdów.cpp**.

void RejestrPojazdow::zapiszPojazdyTypu (const std::string & typ, const std::string & nazwaPliku)

Zapisuje pojazdy określonego typu do pliku.

Parametry

typ	Typ pojazdów do zapisania.
nazwaPliku	Nazwa pliku do zapisu.

Definicja w linii **69** pliku **RejestrPojazdów.cpp**.

void RejestrPojazdow::zapiszPojazdyWlasciciela (const std::string & pesel, const std::string & nazwaPliku)

Zapisuje pojazdy należące do danego właściciela do pliku.

Parametry

pesel	Numer PESEL właściciela.
nazwaPliku	Nazwa pliku do zapisu.

Definicja w linii **82** pliku **RejestrPojazdów.cpp**.

bool RejestrPojazdow::zmienWlasciciela (const std::string & nrRejestracyjny, const Wlasciciel & nowyWlasciciel)

Zmienia właściciela pojazdu.

Parametry

nrRejestracyjny	Numer rejestracyjny pojazdu.
nowyWlasciciel	Nowy właściciel pojazdu.

Zwraca

true jeśli właściciel został zmieniony, false jeśli pojazd nie istnieje.

Definicja w linii **54** pliku **RejestrPojazdów.cpp**.

*Wlasciciel * RejestrPojazdow::znajdzWlasciciela (const std::string & pesel)*

Znajduje właściciela na podstawie numeru PESEL.

Parametry

pesel	Numer PESEL właściciela.

Zwraca

Wskaźnik na obiekt typu **Wlasciciel**, jeśli znaleziono, w przeciwnym razie nullptr.

Definicja w linii **32** pliku **RejestrPojazdów.cpp**.

Dokumentacja atrybutów składowych

std::string RejestrPojazdow::nazwaPliku

Nazwa pliku do wczytania danych.

Definicja w linii **35** pliku **RejestrPojazdów.h**.

std::vector<Pojazd> RejestrPojazdow::pojazdy* `[private]`

Wektor wskaźników na obiekty typu **Pojazd**.

Definicja w linii **24** pliku **RejestrPojazdów.h**.

std::vector<Wlasciciel> RejestrPojazdow::wlasciciele `[private]`

Wektor obiektów typu **Wlasciciel**.

Definicja w linii **29** pliku **RejestrPojazdów.h**.

Dokumentacja klasy Wlasciciel

`#include <Wlasciciel.h>`

Metody publiczne

- **Wlasciciel** (const string &**imie**, const string &**nazwisko**, const string &**pesel**)
- void **printInfo** () const
- string **getImie** () const
- string **getNazwisko** () const
- string **getPesel** () const

Atrybuty prywatne

- string **imie**
- string **nazwisko**
- string **pesel**

Przyjaciele

- ostream & **operator<<** (ostream &os, const **Wlasciciel** &wlasciciel)

Opis szczegółowy

Definicja w linii **4** pliku **Wlasciciel.h**.

Dokumentacja konstruktora i destruktora

Wlasciciel::Wlasciciel (const string & imie*, const string &* nazwisko*, const string &* pesel*)*

Definicja w linii **4** pliku **Wlasciciel.cpp**.

Dokumentacja funkcji składowych

string Wlasciciel::getImie () const

Definicja w linii **15** pliku **Wlasciciel.cpp**.

string Wlasciciel::getNazwisko () const

Definicja w linii **12** pliku **Wlasciciel.cpp**.

string Wlasciciel::getPesel () const

Definicja w linii **9** pliku **Wlasciciel.cpp**.

void Wlasciciel::printInfo () const

Definicja w linii **6** pliku **Wlasciciel.cpp**.

Dokumentacja przyjaciół i funkcji związanych

ostream & operator<< (ostream & os, *const Wlasciciel &* wlasciciel) `[friend]`

Definicja w linii **18** pliku **Wlasciciel.cpp**.

Dokumentacja atrybutów składowych

string Wlasciciel::imie `[private]`

Definicja w linii **7** pliku **Wlasciciel.h**.

string Wlasciciel::nazwisko `[private]`

Definicja w linii **8** pliku **Wlasciciel.h**.

string Wlasciciel::pesel `[private]`

Definicja w linii **9** pliku **Wlasciciel.h**.

Dokumentacja klasy Pojazd

Klasa reprezentująca pojazd.

`#include <Pojazd.h>`

Metody publiczne

- **Pojazd** (const std::string &**typ**, const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**)
  *Konstruktor klasy **Pojazd**.*

- virtual void **printInfo** () const
  Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

- virtual string **getTyp** () const
  Zwraca typ pojazdu.

- virtual string **getMarka** () const
  Zwraca markę pojazdu.

- virtual string **getNazwa** () const
  Zwraca nazwę pojazdu.

- virtual string **getnrRejestracyjny** () const
  Zwraca numer rejestracyjny pojazdu.

- void **ustawWlasciciela** (const **Wlasciciel** &nowyWlasciciel)
  Ustawia nowego właściciela pojazdu.

- const **Wlasciciel** & **getWlasciciel** () const
  Zwraca właściciela pojazdu.

- virtual void **zapiszDoPliku** (std::ofstream &plik) const
  Zapisuje dane pojazdu do podanego strumienia plikowego.

- virtual void **zapiszKoniec** (std::ofstream &plik) const
  Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Atrybuty chronione

- std::string **typ**
  *Typ pojazdu (**SamochodOsobowy**, motocykl, **SamochodCiezarowy**).*

- std::string **marka**
  Marka pojazdu.

- std::string **nazwa**
  Nazwa pojazdu.

- std::string **nrRejestracyjny**
  Numer rejestracyjny pojazdu.

- **Wlasciciel wlasciciel**
  Właściciel pojazdu.

Opis szczegółowy

Klasa reprezentująca pojazd.
Definicja w linii **12** pliku **Pojazd.h**.

Dokumentacja konstruktora i destruktora

Pojazd::Pojazd (const std::string & typ, *const std::string &* marka, *const std::string &* nazwa, *const std::string &* nrRejestracyjny, *const Wlasciciel &* wlasciciel*)*

Konstruktor klasy **Pojazd**.

Parametry

typ	Typ pojazdu.
marka	Marka pojazdu.
nazwa	Nazwa pojazdu.
nrRejestracyjny	Numer rejestracyjny pojazdu.
wlasciciel	Właściciel pojazdu.

Definicja w linii **6** pliku **Pojazd.cpp**.

Dokumentacja funkcji składowych

string Pojazd::getMarka () const `[virtual]`

Zwraca markę pojazdu.

Zwraca

Marka pojazdu.

Definicja w linii **19** pliku **Pojazd.cpp**.

string Pojazd::getNazwa () const `[virtual]`

Zwraca nazwę pojazdu.

Zwraca

Nazwa pojazdu.

Definicja w linii **22** pliku **Pojazd.cpp**.

string Pojazd::getnrRejestracyjny () const `[virtual]`

Zwraca numer rejestracyjny pojazdu.

Zwraca

Numer rejestracyjny pojazdu.

Definicja w linii **25** pliku **Pojazd.cpp**.

string Pojazd::getTyp () const `[virtual]`

Zwraca typ pojazdu.

Zwraca

Typ pojazdu.

Definicja w linii **28** pliku **Pojazd.cpp**.

const Wlasciciel & Pojazd::getWlasciciel () const

Zwraca właściciela pojazdu.

Zwraca

Referencja do właściciela pojazdu.

Definicja w linii **34** pliku **Pojazd.cpp**.

void Pojazd::printInfo () const `[virtual]`

Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

Reimplementowana w **SamochodCiezarowy** ,**Motocykl** i **SamochodOsobowy**.

Definicja w linii **10** pliku **Pojazd.cpp**.

void Pojazd::ustawWlasciciela (const Wlasciciel & nowyWlasciciel*)*

Ustawia nowego właściciela pojazdu.

Parametry

nowyWlasciciel	Nowy właściciel pojazdu.

Definicja w linii **31** pliku **Pojazd.cpp**.

void Pojazd::zapiszDoPliku (std::ofstream & plik*) const* `[virtual]`

Zapisuje dane pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje podstawowe dane pojazdu do podanego strumienia plikowego. Każda klasa pochodna zapisuje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dane.

Reimplementowana w **SamochodCiezarowy** , **Motocykl** i **SamochodOsobowy**.

Definicja w linii **38** pliku **Pojazd.cpp**.

void Pojazd::zapiszKoniec (std::ofstream & plik) const [virtual]

Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje do pliku koncowego aktualny stan programu. Każda klasa pochodna dodaje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dodatkowe informacje.

Reimplementowana w **SamochodCiezarowy**, **Motocykl**) i **SamochodOsobowy**.

Definicja w linii **46** pliku **Pojazd.cpp**.

Dokumentacja atrybutów składowych

std::string Pojazd::marka [protected]

Marka pojazdu.

Definicja w linii **16** pliku **Pojazd.h**.

std::string Pojazd::nazwa [protected]

Nazwa pojazdu.

Definicja w linii **17** pliku **Pojazd.h**.

std::string Pojazd::nrRejestracyjny [protected]

Numer rejestracyjny pojazdu.

Definicja w linii **18** pliku **Pojazd.h**.

std::string Pojazd::typ [protected]

Typ pojazdu (**SamochodOsobowy**, motocykl, **SamochodCiezarowy**).

Definicja w linii **15** pliku **Pojazd.h**.

Wlasciciel Pojazd::wlasciciel [protected]

Właściciel pojazdu.

Definicja w linii **19** pliku **Pojazd.h**.

Dokumentacja klasy SamochodCiezarowy

```
#include <ciezarowy.h>
```

Metody publiczne

- **SamochodCiezarowy** (const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**, int **ladownosc**)
- void **printInfo** () const override
  Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

- int **getLadownosc** () const
- void **zapiszDoPliku** (std::ofstream &plik) const
  Zapisuje dane pojazdu do podanego strumienia plikowego.

- void **zapiszKoniec** (std::ofstream &plik) const
  Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metody publiczne dziedziczone z Pojazd

- **Pojazd** (const std::string &**typ**, const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**)
  *Konstruktor klasy **Pojazd**.*

- virtual string **getTyp** () const
  Zwraca typ pojazdu.

- virtual string **getMarka** () const
  Zwraca markę pojazdu.

- virtual string **getNazwa** () const
  Zwraca nazwę pojazdu.

- virtual string **getnrRejestracyjny** () const
  Zwraca numer rejestracyjny pojazdu.

- void **ustawWlasciciela** (const **Wlasciciel** &nowyWlasciciel)
  Ustawia nowego właściciela pojazdu.

- const **Wlasciciel** & **getWlasciciel** () const
  Zwraca właściciela pojazdu.

Atrybuty prywatne

- int **ladownosc**

Dodatkowe Dziedziczone Składowe

Atrybuty chronione dziedziczone z Pojazd

- std::string **typ**
  *Typ pojazdu (**SamochodOsobowy**, motocykl, **SamochodCiezarowy**).*

- std::string **marka**
  Marka pojazdu.

- std::string **nazwa**
  Nazwa pojazdu.

- std::string **nrRejestracyjny**
  Numer rejestracyjny pojazdu.

- **Wlasciciel wlasciciel**
  Właściciel pojazdu.

Opis szczegółowy

Definicja w linii **4** pliku **ciezarowy.h**.

Dokumentacja konstruktora i destruktora

SamochodCiezarowy::SamochodCiezarowy (const std::string & marka*, const std::string &* nazwa*, const std::string &* nrRejestracyjny*, const Wlasciciel &* wlasciciel*, int* ladownosc*)*

Definicja w linii **3** pliku **ciezarowy.cpp**.

Dokumentacja funkcji składowych

int SamochodCiezarowy::getLadownosc () const

Definicja w linii **11** pliku **ciezarowy.cpp**.

void SamochodCiezarowy::printInfo () const `[override]`, `[virtual]`

Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **6** pliku **ciezarowy.cpp**.

void SamochodCiezarowy::zapiszDoPliku (std::ofstream & plik*) const* `[virtual]`

Zapisuje dane pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje podstawowe dane pojazdu do podanego strumienia plikowego. Każda klasa pochodna zapisuje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dane.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **14** pliku **ciezarowy.cpp**.

void SamochodCiezarowy::zapiszKoniec (std::ofstream & plik) const [virtual]

Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje do pliku koncowego aktualny stan programu. Każda klasa pochodna dodaje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.15*).

Definicja w linii **19** pliku **ciezarowy.cpp**.

Dokumentacja atrybutów składowych

int SamochodCiezarowy::ladownosc [private]

Definicja w linii **7** pliku **ciezarowy.h**.

Dokumentacja klasy SamochodOsobowy

```
#include <osobowy.h>
```

Metody publiczne

- **SamochodOsobowy** (const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**, int **iloscPasazerow**)
- void **printInfo** () const override
  Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

- int **getIloscPasazerow** () const
- void **zapiszDoPliku** (std::ofstream &plik) const
  Zapisuje dane pojazdu do podanego strumienia plikowego.

- void **zapiszKoniec** (std::ofstream &plik) const
  Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metody publiczne dziedziczone z Pojazd

- **Pojazd** (const std::string &**typ**, const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**)
  *Konstruktor klasy **Pojazd**.*

- virtual string **getTyp** () const
  Zwraca typ pojazdu.

- virtual string **getMarka** () const
  Zwraca markę pojazdu.

- virtual string **getNazwa** () const
  Zwraca nazwę pojazdu.

- virtual string **getnrRejestracyjny** () const
  Zwraca numer rejestracyjny pojazdu.

- void **ustawWlasciciela** (const **Wlasciciel** &nowyWlasciciel)
  Ustawia nowego właściciela pojazdu.

- const **Wlasciciel** & **getWlasciciel** () const
  Zwraca właściciela pojazdu.

Atrybuty prywatne

- int **iloscPasazerow**

Dodatkowe Dziedziczone Składowe

Atrybuty chronione dziedziczone z Pojazd

- std::string **typ**

*Typ pojazdu (**SamochodOsobowy**, motocykl, **SamochodCiezarowy**).*

- std::string **marka**
  Marka pojazdu.

- std::string **nazwa**
  Nazwa pojazdu.

- std::string **nrRejestracyjny**
  Numer rejestracyjny pojazdu.

- **Wlasciciel wlasciciel**
  Właściciel pojazdu.

Opis szczegółowy

Definicja w linii **3** pliku **osobowy.h**.

Dokumentacja konstruktora i destruktora

SamochodOsobowy::SamochodOsobowy (const std::string & marka*, const std::string &* nazwa*, const std::string &* nrRejestracyjny*, const Wlasciciel &* wlasciciel*, int* iloscPasazerow*)*

Definicja w linii **3** pliku **osobowy.cpp**.

Dokumentacja funkcji składowych

int SamochodOsobowy::getIloscPasazerow () const

Definicja w linii **11** pliku **osobowy.cpp**.

void SamochodOsobowy::printInfo () const `[override], [virtual]`

Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **6** pliku **osobowy.cpp**.

void SamochodOsobowy::zapiszDoPliku (std::ofstream & plik*) const* `[virtual]`

Zapisuje dane pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje podstawowe dane pojazdu do podanego strumienia plikowego. Każda klasa pochodna zapisuje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dane.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **14** pliku **osobowy.cpp**.

void SamochodOsobowy::zapiszKoniec (std::ofstream & plik*) const* `[virtual]`

Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje do pliku koncowego aktualny stan programu. Każda klasa pochodna dodaje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.15*).

Definicja w linii **18** pliku **osobowy.cpp**.

Dokumentacja atrybutów składowych

int SamochodOsobowy::iloscPasazerow `[private]`

Definicja w linii **6** pliku **osobowy.h**.

Dokumentacja klasy Motocykl

`#include <Motocykl.h>`

Metody publiczne

- **Motocykl** (const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**, int **pojemnoscSilnika**, int **mocSilnika**)
- void **printInfo** () const override
  Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

- int **getPojemnoscSilnika** () const
- int **getMocSilnika** () const
- void **zapiszDoPliku** (std::ofstream &plik) const
  Zapisuje dane pojazdu do podanego strumienia plikowego.

- void **zapiszKoniec** (std::ofstream &plik) const
  Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metody publiczne dziedziczone z Pojazd

- **Pojazd** (const std::string &**typ**, const std::string &**marka**, const std::string &**nazwa**, const std::string &**nrRejestracyjny**, const **Wlasciciel** &**wlasciciel**)
  *Konstruktor klasy **Pojazd**.*

- virtual string **getTyp** () const
  Zwraca typ pojazdu.

- virtual string **getMarka** () const
  Zwraca markę pojazdu.

- virtual string **getNazwa** () const
  Zwraca nazwę pojazdu.

- virtual string **getnrRejestracyjny** () const
  Zwraca numer rejestracyjny pojazdu.

- void **ustawWlasciciela** (const **Wlasciciel** &nowyWlasciciel)
  Ustawia nowego właściciela pojazdu.

- const **Wlasciciel** & **getWlasciciel** () const
  Zwraca właściciela pojazdu.

Atrybuty prywatne

- int **pojemnoscSilnika**
- int **mocSilnika**

Dodatkowe Dziedziczone Składowe

Atrybuty chronione dziedziczone z Pojazd

- std::string **typ**
  *Typ pojazdu (**SamochodOsobowy**, motocykl, **SamochodCiezarowy**).*

- std::string **marka**
  Marka pojazdu.

- std::string **nazwa**
  Nazwa pojazdu.

- std::string **nrRejestracyjny**
  Numer rejestracyjny pojazdu.

- **Wlasciciel wlasciciel**
  Właściciel pojazdu.

Opis szczegółowy

Definicja w linii **3** pliku **Motocykl.h**.

Dokumentacja konstruktora i destruktora

Motocykl::Motocykl (const std::string & marka*, const std::string &* nazwa*, const std::string &* nrRejestracyjny*, const Wlasciciel &* wlasciciel*, int* pojemnoscSilnika*, int* mocSilnika*)*

Definicja w linii **3** pliku **Motocykl.cpp**.

Dokumentacja funkcji składowych

int Motocykl::getMocSilnika () const

Definicja w linii **16** pliku **Motocykl.cpp**.

int Motocykl::getPojemnoscSilnika () const

Definicja w linii **12** pliku **Motocykl.cpp**.

void Motocykl::printInfo () const `[override]`, `[virtual]`

Wyświetla informacje o pojeździe. Metoda wirtualna - kazda klasa dziedzicząca wyswietla swoje dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **6** pliku **Motocykl.cpp**.

void Motocykl::zapiszDoPliku (std::ofstream & plik*) const* `[virtual]`

Zapisuje dane pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje podstawowe dane pojazdu do podanego strumienia plikowego. Każda klasa pochodna zapisuje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dane.

Reimplementowana z **Pojazd** (*str.14*).

Definicja w linii **19** pliku **Motocykl.cpp**.

void Motocykl::zapiszKoniec (std::ofstream & plik) const [virtual]

Zapisuje dodatkowe informacje pojazdu do podanego strumienia plikowego.

Metoda wirtualna, która zapisuje do pliku koncowego aktualny stan programu. Każda klasa pochodna dodaje swoje dodatkowe informacje.

Parametry

plik	Strumień plikowy, do którego zapisywane są dodatkowe informacje.

Reimplementowana z **Pojazd** (*str.15*).

Definicja w linii **25** pliku **Motocykl.cpp**.

Dokumentacja atrybutów składowych

int Motocykl::mocSilnika [private]

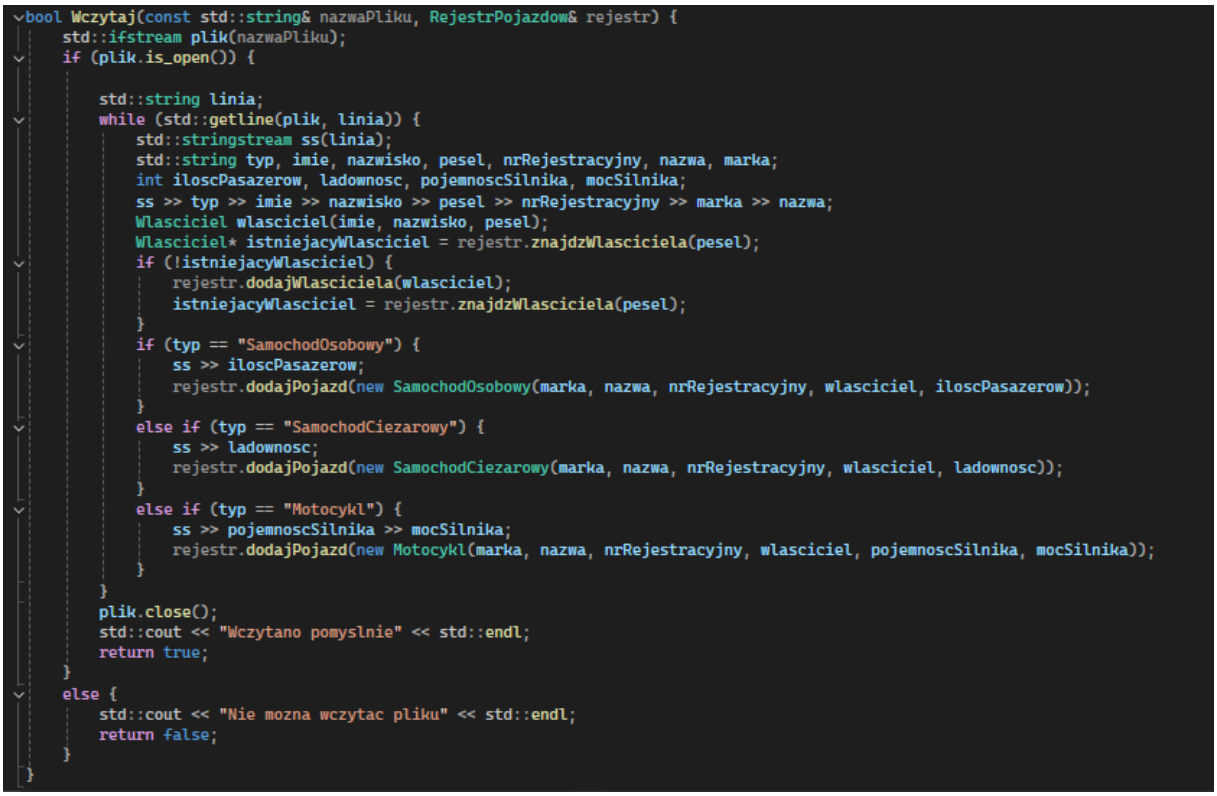
Definicja w linii **7** pliku **Motocykl.h**.

int Motocykl::pojemnoscSilnika [private]

Definicja w linii **6** pliku **Motocykl.h**.

4. Najbardziej istotne fragment kodu.

Wczytywanie do pliku- wczytuje dane odpowiednio przygotowanego pliku tekstowego do rejestru:



```
bool Wczytaj(const std::string& nazwaPliku, RejestrPojazdow& rejestr) {
    std::ifstream plik(nazwaPliku);
    if (plik.is_open()) {

        std::string linia;
        while (std::getline(plik, linia)) {
            std::stringstream ss(linia);
            std::string typ, imie, nazwisko, pesel, nrRejestracyjny, nazwa, marka;
            int iloscPasazerow, ladownosc, pojemnoscSilnika, mocSilnika;
            ss >> typ >> imie >> nazwisko >> pesel >> nrRejestracyjny >> marka >> nazwa;
            Wlasciciel wlasciciel(imie, nazwisko, pesel);
            Wlasciciel* istniejacyWlasciciel = rejestr.znajdzWlasciciela(pesel);
            if (!istniejacyWlasciciel) {
                rejestr.dodajWlasciciela(wlasciciel);
                istniejacyWlasciciel = rejestr.znajdzWlasciciela(pesel);
            }
            if (typ == "SamochodOsobowy") {
                ss >> iloscPasazerow;
                rejestr.dodajPojazd(new SamochodOsobowy(marka, nazwa, nrRejestracyjny, wlasciciel, iloscPasazerow));
            }
            else if (typ == "SamochodCiezarowy") {
                ss >> ladownosc;
                rejestr.dodajPojazd(new SamochodCiezarowy(marka, nazwa, nrRejestracyjny, wlasciciel, ladownosc));
            }
            else if (typ == "Motocykl") {
                ss >> pojemnoscSilnika >> mocSilnika;
                rejestr.dodajPojazd(new Motocykl(marka, nazwa, nrRejestracyjny, wlasciciel, pojemnoscSilnika, mocSilnika));
            }
        }
        plik.close();
        std::cout << "Wczytano pomyslnie" << std::endl;
        return true;
    }
    else {
        std::cout << "Nie mozna wczytac pliku" << std::endl;
        return false;
    }
}
```

Zapisywanie po zakończeniu programu- wykorzystujemy jeden z głównych założeń projektu polimorfizm wywołujący odpowiednią metodę dla danego obiektu tak aby stworzyć sformatowany plik tak, aby można było go uruchomić przy ponownym uruchomieniu programu:

```cpp
bool Zapisz(const std::string& nazwaPliku, const RejestrPojazdow& rejestr) {
    std::ofstream plik(nazwaPliku);
    if (plik.is_open()) {
        for (const auto& pojazd : rejestr.getPojazdy()) {
            pojazd->zapiszKoniec(plik);
        }
        plik.close();
        return true;
    }
    else {
        std::cout << "Nie mozna zapisac pliku" << std::endl;
        return false;
    }
}
```

Użycie zaprzyjaźnionego do klasy Właściciel operatora strumieniowego do wypisywania właścicieli na ekran:

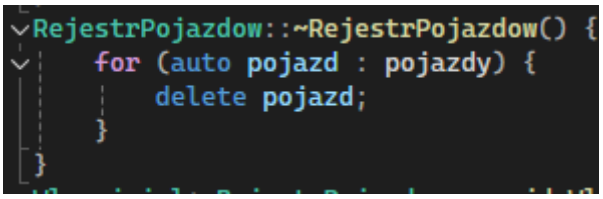
```cpp
ostream& operator<<(ostream& os, const Wlasciciel& wlasciciel) {
    os << "Dane: " << wlasciciel.imie << " " << wlasciciel.nazwisko << " Pesel: " << wlasciciel.pesel;
    return os;
}
```

Wirtualna metoda printInfo do wypisywania danych na temat pojazdu (zdjęcie drugie pokazuje użycie metody dla przykładowej klasy pochodnej:

```cpp
void Pojazd::printInfo() const {
    cout << "Typ: " << typ << endl;
    cout << "Marka: " << marka << endl;
    cout << "Nazwa: " << nazwa << endl;
    cout << "Numer rejestracyjny: " << nrRejestracyjny << endl;
    cout << "Wlasciciel: ";
    wlasciciel.printInfo();
}
```

```cpp
void SamochodOsobowy::printInfo() const {
    Pojazd::printInfo();
    std::cout << "Ilosc miejsc: " << iloscPasazerow << std::endl;
    std::cout << std::endl;
}
```

Destruktor klasy rejestr pojazdów zwalnia pamięć zajmowaną przez dynamiczne obiekty pojazdów



5. Wszystkie kody źródłowe:

Funkcje.cpp

```
00001 #include "funkcje.h"
00002 #include "ciezarowy.h"
00003 #include "Motocykl.h"
00004 #include "Pojazd.h"
00005 #include "osobowy.h"
00006 #include "Wlasciciel.h"
00007 #include "RejestrPojazdów.h"
00008 #include <iostream>
00009 #include <fstream>
00010 #include <string>
00011 #include <sstream>
00012
00013
00014 bool Wczytaj(const std::string& nazwaPliku, RejestrPojazdow& rejestr) {
00015     std::ifstream plik(nazwaPliku);
00016     if (plik.is_open()) {
00017
00018         std::string linia;
00019         while (std::getline(plik, linia)) {
00020             std::stringstream ss(linia);
00021             std::string typ, imie, nazwisko, pesel, nrRejestracyjny, nazwa, marka;
00022             int iloscPasazerow, ladownosc, pojemnoscSilnika, mocSilnika;
00023             ss >> typ >> imie >> nazwisko >> pesel >> nrRejestracyjny >> marka >> nazwa;
00024             Wlasciciel wlasciciel(imie, nazwisko, pesel);
00025             Wlasciciel* istniejacyWlasciciel = rejestr.znajdzWlasciciela(pesel);
00026             if (!istniejacyWlasciciel) {
00027                 rejestr.dodajWlasciciela(wlasciciel);
00028                 istniejacyWlasciciel = rejestr.znajdzWlasciciela(pesel);
00029             }
00030             if (typ == "SamochodOsobowy") {
00031                 ss >> iloscPasazerow;
00032                 rejestr.dodajPojazd(new SamochodOsobowy(marka, nazwa, nrRejestracyjny,
wlasciciel, iloscPasazerow));
00033             }
00034             else if (typ == "SamochodCiezarowy") {
00035                 ss >> ladownosc;
00036                 rejestr.dodajPojazd(new SamochodCiezarowy(marka, nazwa, nrRejestracyjny,
wlasciciel, ladownosc));
00037             }
00038             else if (typ == "Motocykl") {
00039                 ss >> pojemnoscSilnika >> mocSilnika;
00040                 rejestr.dodajPojazd(new Motocykl(marka, nazwa, nrRejestracyjny,
wlasciciel, pojemnoscSilnika, mocSilnika));
00041             }
00042         }
00043         plik.close();
00044         std::cout << "Wczytano pomyslnie" << std::endl;
00045         return true;
00046     }
00047     else {
00048         std::cout << "Nie mozna wczytac pliku" << std::endl;
00049         return false;
00050     }
00051 }
00052 bool Zapisz(const std::string& nazwaPliku, const RejestrPojazdow& rejestr) {
00053     std::ofstream plik(nazwaPliku);
00054     if (plik.is_open()) {
00055         for (const auto& pojazd : rejestr.getPojazdy()) {
00056             pojazd->zapiszKoniec(plik);
00057         }
00058         plik.close();
00059         return true;
00060     }
00061     else {
00062         std::cout << "Nie mozna zapisac pliku" << std::endl;
00063         return false;
00064     }
00065 }
00066
```

```
00067 void wyswietlMenu() {
00068     std::cout << "Menu:" << std::endl;
00069     std::cout << "1. Dodaj nowy pojazd" << std::endl;
00070     std::cout << "2. Usun pojazd" << std::endl;
00071     std::cout << "3. Zmien wlasciciela pojazdu" << std::endl;
00072     std::cout << "4. Wypisz wszystkie pojazdy" << std::endl;
00073     std::cout << "5. Zapisz typ pojazdu do pliku" << std::endl;
00074     std::cout << "6. Wypisz wszystkich wlascicieli" << std::endl;
00075     std::cout << "7. Zapisz pojazdy wybranego wlasciciela do pliku " << std::endl;
00076     std::cout << "8. Wyswietl menu" << std::endl;
00077     std::cout << "0. Zakoncz program" << std::endl;
00078 }
00079 void menu(RejestrPojazdow& rejestr) {
00080     int wybor = -1;
00081     while (wybor != 0) {
00082         std::cout << "Wybierz opcje: ";
00083         std::cin >> wybor;
00084 
00085         while (!std::cin) {
00086             std::cout << "Bledne dane wejsciowe. Wprowadzono niepoprawny znak." <<
std::endl;
00087             std::cin.clear();
00088             std::cin.ignore(1000, '\n');
00089             std::cout << "Wybierz opcje: ";
00090             std::cin >> wybor;
00091         }
00092         switch (wybor) {
00093         case 1: {
00094             string marka, nazwa, nrRejestracyjny, pesel;
00095             std::cout << "Podaj marke (1 wyraz- wiecej nie jest brane pod uwage): ";
00096             std::cin >> marka;
00097             std::cin.clear();
00098             std::cin.ignore(1000, '\n');
00099             std::cout << "Podaj nazwe (1 wyraz- wiecej nie jest brane pod uwage): ";
00100             std::cin >> nazwa;
00101             std::cin.clear();
00102             std::cin.ignore(1000, '\n');
00103             bool poprawnyNr = false;
00104             while (!poprawnyNr) {
00105                 cout << "Podaj numer rejestracyjny: ";
00106                 cin >> nrRejestracyjny;
00107                 poprawnyNr = true;
00108                 for (auto pojazd : rejestr.getPojazdy()) {
00109                     if (pojazd->getnrRejestracyjny() == nrRejestracyjny) {
00110                         poprawnyNr = false;
00111                         std::cout << "Pojazd o takim numerze juz istnieje. Sprobuj
ponownie." << std::endl;
00112                         std::cin.clear();
00113                         std::cin.ignore(1000, '\n');
00114                         break;
00115                     }
00116                 }
00117             }
00118             std::cin.clear();
00119             std::cin.ignore(1000, '\n');
00120             std::cout << "Lista wlascicieli w systemie: " << std::endl;
00121             rejestr.wypiszWlascicieli();
00122             bool poprawnyPesel = false;
00123             while (!poprawnyPesel) {
00124                 std::cout << "Podaj pesel wlasciciela pojazdu (11cyfr): ";
00125                 std::cin >> pesel;
00126                 if (pesel.size() == 11) {
00127                     poprawnyPesel = true;
00128                     for (char c : pesel) {
00129                         if (!isdigit(c)) {
00130                             poprawnyPesel = false;
00131                             break;
00132                         }
00133                     }
00134                 }
00135                 if (!poprawnyPesel) {
00136                     std::cout << "Podano bledny pesel. Sprobuj ponownie." << std::endl;
00137                     std::cin.clear();
00138                     std::cin.ignore(1000, '\n');
00139                 }
00140             }
00141             Wlasciciel* wlasciciel = nullptr;
```

```
00142             if (rejestr.znajdzWlasciciela(pesel) == nullptr) {
00143                 string imie, nazwisko;
00144                 std::cout << "Nie znaleziono wlasciciela o takim peselu w systemie -
tworzymy nowego." << std::endl;
00145                 bool poprawneImie = false;
00146                 while (!poprawneImie) {
00147                     std::cout << "Podaj imie: ";
00148                     std::cin >> imie;
00149                     poprawneImie = true;
00150                     for (char c : imie) {
00151                         if (!isalpha(c)) {
00152                             poprawneImie = false;
00153                             std::cout << "Imie nie moze zawierac cyfr. Sprobuj
ponownie." << std::endl;
00154                             break;
00155                         }
00156                     }
00157                 }
00158                 bool poprawneNazwisko = false;
00159                 while (!poprawneNazwisko) {
00160                     std::cout << "Podaj nazwisko: ";
00161                     std::cin >> nazwisko;
00162                     poprawneNazwisko = true;
00163                     for (char c : nazwisko) {
00164                         if (!isalpha(c)) {
00165                             poprawneNazwisko = false;
00166                             std::cout << "Nazwisko nie moze zawierac cyfr. Sprobuj
ponownie." << std::endl;
00167                             break;
00168                         }
00169                     }
00170                 }
00171                 Wlasciciel nowyWlasciciel(imie, nazwisko, pesel);
00172                 rejestr.dodajWlasciciela(nowyWlasciciel);
00173                 wlasciciel = rejestr.znajdzWlasciciela(pesel);
00174
00175             }
00176             else {
00177                 wlasciciel = rejestr.znajdzWlasciciela(pesel);
00178             }
00179             std::cout << "Jakiego typu pojazd chcesz dodac: " << std::endl;
00180             std::cout << "1. Samochod osobowy" << std::endl;
00181             std::cout << "2. Motocykl" << std::endl;
00182             std::cout << "3. Samochod ciezarowy" << std::endl;
00183             int typpojazdu=-1;
00184             std::cin >> typpojazdu;
00185             while (!std::cin || typpojazdu < 1 || typpojazdu > 3) {
00186                 std::cout << "Nieznany typ pojazdu. Sprobuj ponownie." << std::endl;
00187                 std::cin.clear();
00188                 std::cin.ignore(1000, '\n');
00189                 std::cout << "Jakiego typu pojazd chcesz dodac: " << std::endl;
00190                 std::cout << "1. Samochod osobowy" << std::endl;
00191                 std::cout << "2. Motocykl" << std::endl;
00192                 std::cout << "3. Samochod ciezarowy" << std::endl;
00193                 std::cin >> typpojazdu;
00194             }
00195             if (typpojazdu == 1) {
00196                 int liczbaPasazerow;
00197                 std::cout << "Podaj liczbe miejsc dla pasazerow (max 9): ";
00198                 std::cin >> liczbaPasazerow;
00199                 while (!std::cin || liczbaPasazerow<1 || liczbaPasazerow >9) {
00200                     std::cout << "Niepoprawne dane wejsciowe. Sprobuj ponownie" <<
std::endl;
00201                     std::cin.clear();
00202                     std::cin.ignore(1000, '\n');
00203                     std::cout << "Podaj liczbe miejsc dla pasazerow (max 9): ";
00204                     std::cin >> liczbaPasazerow;
00205                 }
00206                 rejestr.dodajPojazd(new SamochodOsobowy(marka, nazwa, nrRejestracyjny,
*wlasciciel, liczbaPasazerow));
00207                 std::cout << "Dodano pojazd" << std::endl;
00208             }
00209             else if (typpojazdu == 2) {
00210                 int pojemnoscSilnika, mocSilnika;
00211                 std::cout << "Podaj pojemnosc silnika: ";
00212                 std::cin >> pojemnoscSilnika;
00213                 while (!std::cin || pojemnoscSilnika <= 0) {
```

```
00214                     std::cout << "Niepoprawne dane wejsciowe. Sprobuj ponownie" <<
std::endl;
00215                     std::cin.clear();
00216                     std::cin.ignore(1000, '\n');
00217                     std::cout << "Podaj pojemnosc silnika: ";
00218                     std::cin >> pojemnoscSilnika;
00219                 }
00220                 std::cout << "Podaj moc silnika: ";
00221                 std::cin >> mocSilnika;
00222                 while (!std::cin || mocSilnika <= 0) {
00223                     std::cout << "Niepoprawne dane wejsciowe. Sprobuj ponownie" <<
std::endl;
00224                     std::cin.clear();
00225                     std::cin.ignore(1000, '\n');
00226                     std::cout << "Podaj moc silnika: ";
00227                     std::cin >> mocSilnika;
00228                 }
00229                 rejestr.dodajPojazd(new Motocykl(marka, nazwa, nrRejestracyjny,
*wlasciciel, pojemnoscSilnika, mocSilnika));
00230                 std::cout << "Dodano pojazd" << std::endl;
00231             }
00232             else if (typpojazdu == 3) {
00233                 int ladownosc;
00234                 std::cout << "Podaj ladownosc: ";
00235                 std::cin >> ladownosc;
00236                 while (!std::cin || ladownosc <= 0) {
00237                     std::cout << "Niepoprawne dane wejsciowe. Sprobuj ponownie" <<
std::endl;
00238                     std::cin.clear();
00239                     std::cin.ignore(1000, '\n');
00240                     std::cout << "Podaj ladownosc: ";
00241                     std::cin >> ladownosc;
00242                 }
00243                 rejestr.dodajPojazd(new SamochodCiezarowy(marka, nazwa, nrRejestracyjny,
*wlasciciel, ladownosc));
00244                 std::cout << "Dodano pojazd" << std::endl;
00245             }
00246             break;
00247         }
00248         case 2: {
00249             string nrRejestracyjny;
00250             std::cout << "Podaj numer rejestracyjny pojazdu do usuniecia: ";
00251             std::cin >> nrRejestracyjny;
00252             rejestr.usunPojazd(nrRejestracyjny);
00253             break;
00254         }
00255         case 3: {
00256             string nrRejestracyjny, pesel, imie, nazwisko;
00257             std::cout << "Podaj numer rejestracyjny pojazdu do zmiany wlasciciela: ";
00258             std::cin >> nrRejestracyjny;
00259             std::cout << "Wlasciciele zapisani w systemie:" << std::endl;
00260             rejestr.wypiszWlascicieli();
00261             bool poprawnyPesel = false;
00262             while (!poprawnyPesel) {
00263                 std::cout << "Podaj pesel wlasciciela pojazdu (11cyfr): ";
00264                 std::cin >> pesel;
00265                 if (pesel.size() == 11) {
00266                     poprawnyPesel = true;
00267                     for (char c : pesel) {
00268                         if (!isdigit(c)) {
00269                             poprawnyPesel = false;
00270                             break;
00271                         }
00272                     }
00273                 }
00274                 if (!poprawnyPesel) {
00275                     std::cout << "Podano bledny pesel. Sprobuj ponownie." << std::endl;
00276                     std::cin.clear();
00277                     std::cin.ignore(1000, '\n');
00278                 }
00279             }
00280             Wlasciciel* nowyWlasciciel = rejestr.znajdzWlasciciela(pesel);
00281             if (!nowyWlasciciel) {
00282                 std::cout << "Nieznaleziono wlasciciela o podanym numerze pesel-
tworzymy nowego.";
00283                 bool poprawneImie = false;
00284                 while (!poprawneImie) {
```

```
00285                     std::cout << "Podaj imie: ";
00286                     std::cin >> imie;
00287                     poprawneImie = true;
00288                     for (char c : imie) {
00289                         if (!isalpha(c)) {
00290                             poprawneImie = false;
00291                             std::cout << "Imie nie moze zawierac cyfr. Sprobuj
ponownie." << std::endl;
00292                             break;
00293                         }
00294                     }
00295                 }
00296                 bool poprawneNazwisko = false;
00297                 while (!poprawneNazwisko) {
00298                     std::cout << "Podaj nazwisko: ";
00299                     std::cin >> nazwisko;
00300                     poprawneNazwisko = true;
00301                     for (char c : nazwisko) {
00302                         if (!isalpha(c)) {
00303                             poprawneNazwisko = false;
00304                             std::cout << "Nazwisko nie moze zawierac cyfr. Sprobuj
ponownie." << std::endl;
00305                             break;
00306                         }
00307                     }
00308                 }
00309                 Wlasciciel nowy(imie, nazwisko, pesel);
00310                 rejestr.dodajWlasciciela(nowy);
00311                 nowyWlasciciel = rejestr.znajdzWlasciciela(pesel);
00312             }
00313             rejestr.zmienWlasciciela(nrRejestracyjny, *nowyWlasciciel);
00314             break;
00315         }case 4: {
00316             rejestr.wyswietlPojazdy();
00317             break;
00318         }case 5: {
00319             string typ, nazwaPliku;
00320             std::cout << "Podaj typ pojazdu: " << std::endl;
00321             std::cout << "1. Samochod osobowy" << std::endl;
00322             std::cout << "2. Motocykl" << std::endl;
00323             std::cout << "3. Samochod ciezarowy" << std::endl;
00324
00325             int typpojazdu = -1;
00326             std::cin >> typpojazdu;
00327             while (!std::cin || typpojazdu < 1 || typpojazdu > 3) {
00328                 std::cout << "Nieznany typ pojazdu. Sprobuj ponownie." << std::endl;
00329                 std::cin.clear();
00330                 std::cin.ignore(1000, '\n');
00331                 std::cout << "Jakiego typu pojazd chcesz dodac: " << std::endl;
00332                 std::cout << "1. Samochod osobowy" << std::endl;
00333                 std::cout << "2. Motocykl" << std::endl;
00334                 std::cout << "3. Samochod ciezarowy" << std::endl;
00335                 std::cin >> typpojazdu;
00336             }
00337             if (typpojazdu == 1) {
00338                 typ = "SamochodOsobowy";
00339                 nazwaPliku = "SamochodOsobowy.txt";
00340             }
00341             else if (typpojazdu == 2) {
00342                 typ = "Motocykl";
00343                 nazwaPliku = "Motocykl.txt";
00344             }
00345             else if (typpojazdu == 3) {
00346                 typ = "SamochodCiezarowy";
00347                 nazwaPliku = "SamochodCiezarowy.txt";
00348             }
00349
00350
00351             rejestr.zapiszPojazdyTypu(typ, nazwaPliku);
00352             std::cout << "Zapisano do pliku: " << nazwaPliku << std::endl;
00353             break;
00354         }case 6: {
00355             rejestr.wypiszWlascicieli();
00356             break;
00357         }case 7: {
00358             string pesel, nazwaPliku;
00359             rejestr.wypiszWlascicieli();
```

```
00360             std::cout << "Podaj pesel wlasciciela, ktorego pojazdy chcesz zapisac: ";
00361             std::cin >> pesel;
00362             Wlasciciel* wlasciciel = rejestr.znajdzWlasciciela(pesel);
00363             if (wlasciciel) {
00364                 nazwaPliku = wlasciciel->getImie() + "_" + wlasciciel->getNazwisko() +
"_pojazdy.txt";
00365                 rejestr.zapiszPojazdyWlasciciela(pesel, nazwaPliku);
00366             }
00367             else {
00368                 std::cout << "Wlasciciel o podanym numerze PESEL nie istnieje." <<
std::endl;
00369             }
00370             break;
00371         }
00372         case 8: {
00373             wyswietlMenu();
00374             break;
00375         }
00376         case 0: {
00377             std::cout << "Koniec programu." << std::endl;
00378             break;
00379         }
00380         default: {
00381             std::cout << "Niepoprawna opcja. Sprobuj ponownie." << std::endl;
00382             break;
00383         }
00384         }
00385     }
00386 }
```

Rejestr pojazdów.cpp

```
00001 #include "RejestrPojazdów.h"
00002 #include <fstream>
00003
00004
00005 void RejestrPojazdow::dodajPojazd(Pojazd* pojazd) {
00006     pojazdy.push_back(pojazd);
00007 }
00008 void RejestrPojazdow::usunPojazd(const std::string& nrRejestracyjny) {
00009     bool usunieto = false;
00010     for (int i=0; i < pojazdy.size(); ++i) {
00011         if (pojazdy[i]->getnrRejestracyjny() == nrRejestracyjny) {
00012             delete pojazdy[i];
00013             pojazdy.erase(pojazdy.begin() + i);
00014             std::cout << "Usunieto pojazd o nr rejestracyjnym: " << nrRejestracyjny <<
std::endl;
00015             usunieto = true;
00016         }
00017     }
00018     if (!usunieto) {
00019         std::cout << "Nieznaleziono pojazdu o podanym numerze rejestracyjnym" <<
std::endl;
00020     }
00021 };
00022 void RejestrPojazdow::wyswietlPojazdy() const {
00023     for (const auto& pojazd : pojazdy) {
00024         pojazd->printInfo();
00025     }
00026 }
00027 RejestrPojazdow::~RejestrPojazdow() {
00028     for (auto pojazd : pojazdy) {
00029         delete pojazd;
00030     }
00031 }
00032 Wlasciciel* RejestrPojazdow::znajdzWlasciciela(const std::string& pesel) {
00033     for (auto& w : wlasciciele) {
00034         if (w.getPesel() == pesel) {
00035             return &w;
00036         }
00037     }
00038     return nullptr;
00039 }
00040 void RejestrPojazdow::wypiszWlascicieli() const {
00041     for (const auto& wlasciciel : wlasciciele) {
00042         std::cout << wlasciciel << std::endl;
```

```
00043     }
00044 }
00045 bool RejestrPojazdow::dodajWlasciciela(const Wlasciciel& wlasciciel) {
00046     for (const auto& w : wlasciciele) {
00047         if (w.getPesel() == wlasciciel.getPesel()) {
00048             return false;
00049         }
00050     }
00051     wlasciciele.push_back(wlasciciel);
00052     return true;
00053 }
00054 bool RejestrPojazdow::zmienWlasciciela(const std::string& nrRejestracyjny, const
Wlasciciel& nowyWlasciciel) {
00055     for (Pojazd* pojazd : pojazdy) {
00056         if (pojazd->getnrRejestracyjny() == nrRejestracyjny) {
00057             pojazd->ustawWlasciciela(nowyWlasciciel);
00058             std::cout << "Wlasciciel pojazdu o numerze rejestracyjnym " <<
nrRejestracyjny << " został zmieniony." << std::endl;
00059             return true;
00060         }
00061     }
00062     std::cout << "Nie znaleziono pojazdu o numerze rejestracyjnym " << nrRejestracyjny
<< ". Sprawdz czy zostal prawidlowo podany." << std::endl;
00063     return false;
00064 }
00065 const std::vector<Pojazd*>& RejestrPojazdow::getPojazdy() const {
00066     return pojazdy;
00067 }
00068
00069 void RejestrPojazdow::zapiszPojazdyTypu(const std::string& typ, const std::string&
nazwaPliku) {
00070     std::ofstream plik(nazwaPliku);
00071     if (!plik.is_open()) {
00072         std::cout << "Nie mozna otworzyc pliku" << std::endl;
00073         return;
00074     }
00075     for (const auto& pojazd : pojazdy) {
00076         if (pojazd->getTyp() == typ) {
00077             pojazd->zapiszDoPliku(plik);
00078             plik << std::endl;
00079         }
00080     }
00081 }
00082 void RejestrPojazdow::zapiszPojazdyWlasciciela(const std::string& pesel, const
std::string& nazwaPliku) {
00083     std::ofstream plik(nazwaPliku);
00084     if (!plik.is_open()) {
00085         std::cout << "Nie mozna otworzyc pliku." << std::endl;
00086         return;
00087     }
00088     Wlasciciel* wlasciciel = znajdzWlasciciela(pesel);
00089     if (!wlasciciel) {
00090         std::cout << "Taki wlasciciel nie istnieje." << std::endl;
00091         return;
00092     }
00093     else {
00094         plik << "Imie: " << wlasciciel->getImie() << std::endl;
00095         plik << "Nazwisko: " << wlasciciel->getNazwisko() << std::endl;
00096         plik << "Pesel: " << wlasciciel->getPesel() << std::endl;
00097         plik << "Pojazdy:" << std::endl;
00098         plik << std::endl;
00099         std::cout << "Zapisano dane do pliku " << nazwaPliku << std::endl;
00100     }
00101     for (const auto& pojazd : pojazdy) {
00102         if (pojazd->getWlasciciel().getPesel() == pesel) {
00103             plik << "Typ: " << pojazd->getTyp() << std::endl;
00104             plik << "Marka: " << pojazd->getMarka() << std::endl;
00105             plik << "Nazwa: " << pojazd->getNazwa() << std::endl;
00106             plik << "Numer rejestracyjny: " << pojazd->getnrRejestracyjny() <<
std::endl;
00107             if (SamochodOsobowy* so = dynamic_cast<SamochodOsobowy*>(pojazd)) {
//rzutowanie *pojazd na SamochodOsobowy* so
00108                 plik << "Ilosc miejsc: " << so->getIloscPasazerow() << std::endl;
00109             }
00110             else if (SamochodCiezarowy* sc = dynamic_cast<SamochodCiezarowy*>(pojazd)) {
00111                 plik << "Maksymalna ladownosc: " << sc->getLadownosc() << std::endl;
00112             }
```

```
00113             else if (Motocykl* m = dynamic_cast<Motocykl*>(pojazd)) {
00114                 plik << "Pojemnosc silnika: " << m->getPojemnoscSilnika() << std::endl;
00115                 plik << "Moc silnika: " << m->getMocSilnika() << std::endl;
00116             }
00117             plik << std::endl;
00118         }
00119     }
00120     plik.close();
00121 }
```

Pojazd.cpp

```
00001 #include "Pojazd.h"
00002 #include "Wlasciciel.h"
00003
00004 using namespace std;
00005
00006 Pojazd::Pojazd(const std::string& typ, const std::string& marka, const std::string&
nazwa, const std::string& nrRejestracyjny, const Wlasciciel& wlasciciel)
00007     : typ(typ), marka(marka), nazwa(nazwa), nrRejestracyjny(nrRejestracyjny),
wlasciciel(wlasciciel) {}
00008
00009
00010 void Pojazd::printInfo() const {
00011     cout << "Typ: " << typ << endl;
00012     cout << "Marka: " << marka << endl;
00013     cout << "Nazwa: " << nazwa << endl;
00014     cout << "Numer rejestracyjny: " << nrRejestracyjny << endl;
00015     cout << "Wlasciciel: ";
00016     wlasciciel.printInfo();
00017
00018 }
00019 string Pojazd::getMarka() const {
00020     return marka;
00021 }
00022 string Pojazd::getNazwa() const {
00023     return nazwa;
00024 }
00025 string Pojazd::getnrRejestracyjny() const {
00026     return nrRejestracyjny;
00027 }
00028 string Pojazd::getTyp() const {
00029     return typ;
00030 }
00031 void Pojazd::ustawWlasciciela(const Wlasciciel& nowyWlasciciel) {
00032     wlasciciel = nowyWlasciciel;
00033 }
00034 const Wlasciciel& Pojazd::getWlasciciel() const {
00035     return wlasciciel;
00036 }
00037
00038 void Pojazd::zapiszDoPliku(std::ofstream& plik) const {
00039     plik << "Typ: " << typ << std::endl;
00040     plik << "Marka: " << marka << std::endl;
00041     plik << "Nazwa: " << nazwa << std::endl;
00042     plik << "Numer rejestracyjny: " << nrRejestracyjny << std::endl;
00043     plik << "Wlasciciel: " << wlasciciel.getImie() << " " << wlasciciel.getNazwisko() <<
" " << wlasciciel.getPesel() << std::endl;
00044 }
00045
00046 void Pojazd::zapiszKoniec(std::ofstream& plik) const
00047 {
00048     plik << typ << " " << wlasciciel.getImie() << " " << wlasciciel.getNazwisko() << " "
<< wlasciciel.getPesel() << " " <<
00049         nrRejestracyjny << " " << marka <<" "<< nazwa <<" ";
00050 }
00051
00052
00053
```

Wlasciciel.cpp

```
00001 #include "Wlasciciel.h"
00002 #include <iostream>
00003
```

```
00004 Wlasciciel::Wlasciciel(const string& imie, const string& nazwisko, const string& pesel)
: imie(imie), nazwisko(nazwisko), pesel(pesel) {}
00005
00006 void Wlasciciel::printInfo() const {
00007     std::cout << imie << " " << nazwisko << " Pesel: " << pesel << std::endl;
00008 }
00009 string Wlasciciel::getPesel() const {
00010     return pesel;
00011 }
00012 string Wlasciciel::getNazwisko() const {
00013     return nazwisko;
00014 }
00015 string Wlasciciel::getImie() const {
00016     return imie;
00017 }
00018 ostream& operator<<(ostream& os, const Wlasciciel& wlasciciel) {
00019     os << "Dane: " << wlasciciel.imie << " " << wlasciciel.nazwisko << " Pesel: " <<
wlasciciel.pesel;
00020     return os;}
```

Osobowy.cpp

```
00001 #include "osobowy.h"
00002
00003 SamochodOsobowy::SamochodOsobowy(const std::string& marka, const std::string& nazwa,
const std::string& nrRejestracyjny, const Wlasciciel& wlasciciel, int iloscPasazerow)
00004     : Pojazd("SamochodOsobowy", marka, nazwa, nrRejestracyjny, wlasciciel),
iloscPasazerow(iloscPasazerow) {}
00005
00006 void SamochodOsobowy::printInfo() const {
00007     Pojazd::printInfo();
00008     std::cout << "Ilosc miejsc: " << iloscPasazerow << std::endl;
00009     std::cout << std::endl;
00010 }
00011 int SamochodOsobowy::getIloscPasazerow() const {
00012     return iloscPasazerow;
00013 }
00014 void SamochodOsobowy::zapiszDoPliku(std::ofstream& plik) const {
00015     Pojazd::zapiszDoPliku(plik);
00016     plik << "Ilosc miejsc: " << iloscPasazerow << std::endl;
00017 }
00018 void SamochodOsobowy::zapiszKoniec(std::ofstream& plik) const
00019 {
00020     Pojazd::zapiszKoniec(plik);
00021     plik << iloscPasazerow << std::endl;
00022 }
```

Motocykl.cpp

```
00001 #include "Motocykl.h"
00002
00003 Motocykl::Motocykl(const std::string& marka, const std::string& nazwa, const
std::string& nrRejestracyjny, const Wlasciciel& wlasciciel, int pojemnoscSilnika, int
mocSilnika)
00004     : Pojazd("Motocykl", marka, nazwa, nrRejestracyjny, wlasciciel),
pojemnoscSilnika(pojemnoscSilnika), mocSilnika(mocSilnika) {}
00005
00006 void Motocykl::printInfo() const {
00007     Pojazd::printInfo();
00008     std::cout << "Pojemnosc silnika: " << pojemnoscSilnika << "cm^3" << std::endl;
00009     std::cout << "Moc silnika: " << mocSilnika << "KM" << std::endl;
00010     std::cout << std::endl;
00011 }
00012 int Motocykl::getPojemnoscSilnika() const {
00013     return pojemnoscSilnika;
00014 }
00015
00016 int Motocykl::getMocSilnika() const {
00017     return mocSilnika;
00018 }
00019 void Motocykl::zapiszDoPliku(std::ofstream& plik) const {
00020     Pojazd::zapiszDoPliku(plik);
00021     plik << "Pojemnosc silnika: " << getPojemnoscSilnika() << std::endl;
00022     plik << "Moc silnika: " << getMocSilnika() << std::endl;
00023 }
```

```
00024
00025 void Motocykl::zapiszKoniec(std::ofstream& plik) const
00026 {
00027     Pojazd::zapiszKoniec(plik);
00028     plik << pojemnoscSilnika << " " << mocSilnika << std::endl;
00029 }
```

Ciezarowy.cpp

```
00001 #include "ciezarowy.h"
00002
00003 SamochodCiezarowy::SamochodCiezarowy(const std::string& marka, const std::string& nazwa,
const std::string& nrRejestracyjny, const Wlasciciel& wlasciciel, int ladownosc)
00004     : Pojazd("SamochodCiezarowy", marka, nazwa, nrRejestracyjny, wlasciciel),
ladownosc(ladownosc) {}
00005
00006 void SamochodCiezarowy::printInfo() const {
00007     Pojazd::printInfo();
00008     std::cout << "Ladownosc: " << ladownosc << " kg" << std::endl;
00009     std::cout << std::endl;
00010 }
00011 int SamochodCiezarowy::getLadownosc() const {
00012     return ladownosc;
00013 }
00014 void SamochodCiezarowy::zapiszDoPliku(std::ofstream& plik) const {
00015     Pojazd::zapiszDoPliku(plik);
00016     plik << "Maksymalna ladownosc: " << getLadownosc() << std::endl;
00017 }
00018
00019 void SamochodCiezarowy::zapiszKoniec(std::ofstream& plik) const
00020 {
00021     Pojazd::zapiszKoniec(plik);
00022     plik << ladownosc << std::endl;
00023 }
```

Main.cpp

```
00001 #include "ciezarowy.h"
00002 #include "Motocykl.h"
00003 #include "Pojazd.h"
00004 #include "osobowy.h"
00005 #include "Wlasciciel.h"
00006 #include "RejestrPojazdów.h"
00007 #include "funkcje.h"
00008
00009
00010
00011
00012 int main(int argc, char* argv[]) {
00013     if (argc != 2) {
00014         std::cerr << "Jako argumentu uzyj nazwy pliku z koncowka .txt" << std::endl;
00015         return 0;
00016     }
00017     RejestrPojazdow rejestr;
00018     rejestr.nazwaPliku = argv[1];
00019
00020
00021     if (!Wczytaj(rejestr.nazwaPliku, rejestr)) {
00022         return 0;
00023     }
00024     else
00025     {   wyswietlMenu();
00026         menu(rejestr);
00027         Zapisz(rejestr.nazwaPliku, rejestr);
00028
00029         return 0;
00030     }
00031 }
```